

An Adaptive Migratory Home Protocol for Software DSM Systems

Chang-Yi Lin, Jiazheng Zhou, and Yeh-Ching Chung

Department of Computer Science

National Tsing-Hua University, Hsinchu, Taiwan 300, ROC

Email: jimmy@sslslab.cs.nthu.edu.tw {jzzhou, ychung}@cs.nthu.edu.tw

Abstract In software distributed shared memory systems, home-based protocols are very good designs because of easy implementation, little memory overheads and good performance. The home-based protocols can be divided into two categories, the fix home protocol and the migratory home protocol. The fix home protocol produces less number of page faults than the migratory home protocol. The migratory home protocols can save the number of diff creating and diff propagation. For an application program, its execution behavior is suitable for the fixed home protocol in some time periods and is suitable for the migratory home protocol in some other time periods. A protocol that can be adapted between the fixed home protocol and the migratory home protocol is important for the performance of an application program. In this paper, we proposed an adaptive migratory home protocol which combines the advantages of the fixed home and the migratory home protocols. In the adaptive migratory home protocol, the home node of a page can change the state of the page according the application behavior. To evaluate the adaptive migratory home protocol, we have implemented the proposed protocol along with the home-based protocol and the migratory home protocol. We tested eight application programs for the three protocols on the same platform. The experimental result shows that the home-based protocol works well for applications with sharing access pattern. The migratory home protocol has good performance for migratory access pattern. The performance of the adaptive migratory home protocol is in between these two protocols and performs well for most of the applications.

Index Terms— Distributed shared memory systems, home-based protocols, home migration.

I. INTRODUCTION

A cluster system consists of a number of nodes connected by an interconnection network. In a cluster system, the physical memory are resides on different nodes. Due to the distributed memory characteristic, in a cluster system, nodes communicate to each other by message passing. This makes the programming on a cluster system difficult.

A distributed shared memory (DSM) system [8] is a mechanism that offers a global shared memory address space for distributed memory computers such as clusters. With the

global shared memory address space, the programming on a DSM system is easier than that on a cluster system. Figure 1 shows a DSM system.

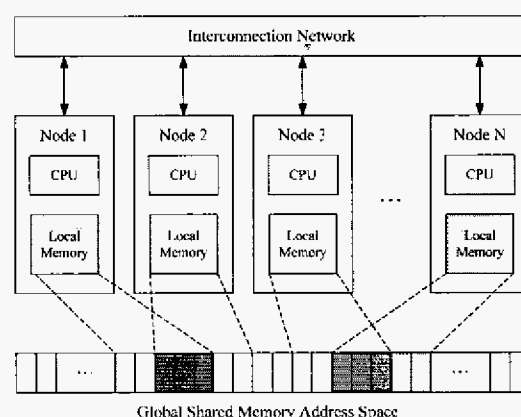


Figure 1: A DSM system.

In general, there are two types of protocols, homeless and home-based [2] [4] [5], to manage the consistency of pages that have more than one copy on different nodes. For the homeless protocol, no home nodes are associated with pages. The idea of home-based protocol is that each page has a home node to maintain the most up-to-date copy of a page. Non-home nodes fetch the whole page from the home node and eagerly send diffs to the home node at release time. But to overcome the poor home assignment problem, the migratory home protocols were proposed [1] [3] [7].

In the migratory home protocol, when a page satisfied certain conditions, the page will be migrated from one home node to another. This page migration behavior may result in a good home assignment at run time. However, if the migration of pages cannot achieve a good home assignment, the performance of the migratory home protocol will be worse than that of the home-based protocol.

In this paper, we propose an adaptive migratory home protocol for LRC model. In this protocol, pages have two modes, MIG and FIX modes, to adapt their behaviors to applications. To evaluate the proposed protocol, we have implemented the proposed protocol along with the home-based

protocol [9] and the migratory home protocol [1] [6] [7] on an 8-node PC cluster. Eight application programs were used as test samples. The result shows that the home-based protocol works well for applications with sharing access pattern. The migratory home protocol has good performance for migratory access pattern. The performance of the adaptive migratory home protocol is in between these two protocols and performs well for most of the applications.

This paper is organized as follows. Section 2 will describe the proposed adaptive migratory home protocol in details. The performance evaluation of the proposed protocol will be given in Section 3. The conclusion of this paper will be given in Section 4.

II. AN ADAPTIVE MIGRATORY HOME PROTOCOL

In this Section, we propose an adaptive migratory home protocol to solve the bad home assignment problem occurred in the fixed home and the migratory home protocols. This protocol combines the advantages of the fixed home and the migratory home protocols. In the adaptive migratory home protocol, the protocol used for a page depends on the execution behavior of an application program. If the fixed home protocol is a good protocol for a page in some time periods, the fixed home protocol is used for the page. In some other time periods, if the migratory home protocol is a better protocol for the page, the protocol will be switched from the fixed home protocol to the migratory home protocol for the page and vice versa.

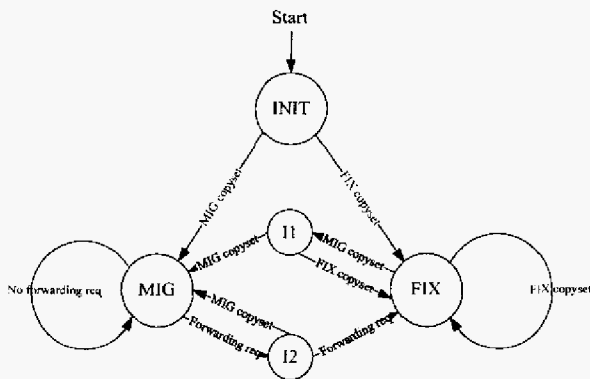


Figure 2: The state transition diagram of the adaptive migratory home protocol.

Figure 2 shows the state transition diagram of the adaptive migratory home protocol. In Figure 2, there are 5 states, INIT, FIX, MIG, I1, and I2. INIT is the initial state. FIX is the state that the fixed home protocol is used for a page. MIG is the state that the migratory home protocol is used for a page. I1 is the intermediate state for a page from the FIX state to the MIG state. I2 is the intermediate state for a page from the MIG state to the FIX state. When an application program is running, each page has its own state and only the home node has the right to change the states of pages that it manages.

At the startup time, all pages are in the INIT state and cannot be migrated. The state of a page will be changed from

the INIT state to the MIG state or the FIX state according to the degree of page sharing by keeping track of the copyset of a page. The degree of a page sharing is high when several copies of the page reside on different nodes at the same time. The degree of a page sharing is low when the page is used by one node. The protocol overheads of pages in the FIX state are lower than those of pages in the MIG states. Therefore, the fixed home protocol provides stable performance for pages having high degree of page sharing. On the other hand, the migratory home protocol deal with the pages that having low degree of page sharing well because of the home effect. A page could be modified by its home node without creating diffs. This is so-called home effect. If pages are modified by their home nodes more frequently, more diff-creating time and communication time will be saved. A good home assignment will result in home nodes modify their managed pages most of time. That is why we change the state of a page according to the degree of page sharing. In the following sections, we describe the conditions that force a page from one state to another.

A. INIT $\rightarrow$ FIX and INIT $\rightarrow$ MIG

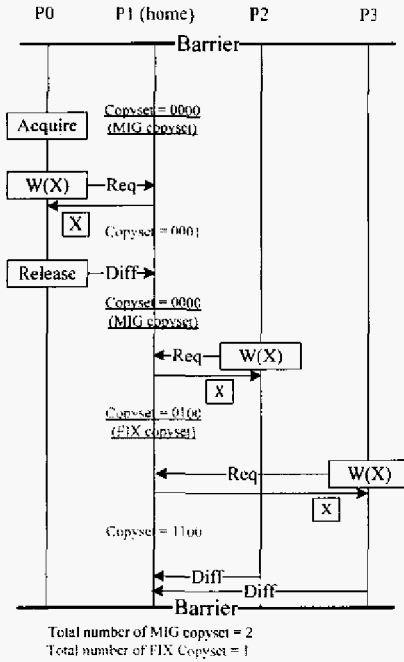
Initially, all pages are in the INIT state and the homes of pages are fixed. For each page, its home node maintains an n -bit copyset to keep track of the copies of the page. If node P_i fetches an up-to-date copy of page x , the i th bit of the copyset of page x will be set to 1 by the home node. If the home node of page x receives a diff from node P_j , it will reset the i th bit of the copyset of page x . By using the copyset, a home node knows how many copies of a page existing on other nodes.

Figure 3 shows an example of keeping track of the copyset of a page. In Figure 3, the home node of page x is fixed to P_1 . When a fault of page x occurs in a critical section on a node, such as P_0 for the given example, P_0 sends a request to P_1 for page x . After P_0 gets page x , the copyset of page x is set to 0001 by P_1 . When P_0 exits the critical section, P_0 sends a diff to P_1 . After receiving the diff from P_0 , P_1 sets the copyset of page x to 0000. When a fault of page x occurs outside of a critical section on a node, such as P_2 and P_3 for the given example, P_2 and P_3 fetch page x from P_1 . After P_2 and P_3 fetch page x , the copyset of page x is set as 1100. The copyset of page x is reset to 0000 by P_1 after passing the next barrier since P_2 and P_3 propagate diffs back to the home node P_1 .

In Figure 3, when P_1 serves the first two requests for page x , the copyset of page x is zero. The copyset of page x is not equal to zero when P_1 serves the third request. When a home node serves a request for a page and the copyset of the page is equal to zero, the copyset of the page is defined as a MIG copyset at that service time. When a home node serves a request for a page and the copyset of the page is not equal to zero, the copyset of the page is defined as a FIX copyset at that service time.

In an interval between two barriers, a home node may serve several requests for a page. The home node records the numbers of the copyset of a page being as a MIG copyset and a FIX copyset, respectively. After the execution of the second barrier of an interval, the home node will compare the total

number of the MIG copyset and the total number of the FIX copyset of a page in the interval. If the total number of the MIG copyset is greater than the total number of the FIX copyset, the page state is changed from the INIT state to the MIG state. Otherwise, the page state is changed from the INIT state to the FIX state.



B. FIX → MIG

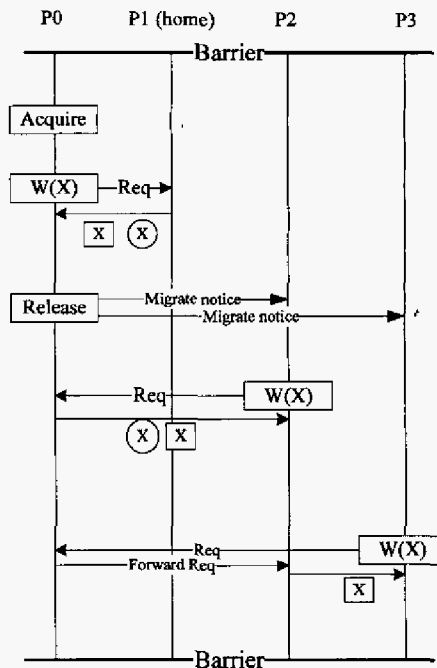
When a page is in the FIX state, the HLRC protocol is used for the page. The home node use the same method described in the INIT state to keep track of the copyset of a page. In an interval, when the home node receives a request for page x , the FIX copyset of page x means that the access pattern of page x may result in page request forwarding if page x is in the MIG state. On the other hand, the MIG copyset means that page x should be migrated to the requester if page x is in the MIG state. After the execution of the second barrier of an interval, the home node will compare the total number of the MIG copyset and the total number of the FIX copyset of a page in the interval. If the total number of the MIG copyset is greater than the total number of the FIX copyset, the page state is changed from the FIX state to the I1 state. Otherwise, the page state is not changed.

For a page in the I1 state, the home node performs the same method described in the INIT state to keep track of the copyset of the page. At a barrier, the state of a page will be change from the I1 state to the MIG state if the total number of the MIG copyset is greater than the total number of the FIX copyset on request. Otherwise, the state of the page is changed from the I1 state to the FIX state. The lifetime of the I1 state is short.

C. MIG → FIX

For a page in the MIG state, the home node uses the same method described in the INIT state to keep track of the copyset of a page. A page can be migrated from one node to another if the home node of a page detects a MIG copyset when it serves requests from other nodes. After the home migration, other nodes should be informed about the home migration. However, sending migration notices for every migration costs too much. The migration notices are sent by the new home node at release time and many migration notices can be combined into one message. Therefore, the overhead is reduced a lot.

However, sending migration notices at release time may result in the request forwarding problem since other nodes may not know the migration. An example of the request forwarding is shown in Figure 4. In Figure 4, the access pattern is the same as that in Figure 3. The difference is that page x is in the INIT state and the MIG state in Figure 3 and Figure 4, respectively. In Figure 3, when P_1 serves the first two requests from P_0 and P_2 , the copyset of page x is equal to zero. The copyset of page x is not equal to zero when P_1 serves the request from P_3 . Therefore, in Figure 4, the request of page x from P_0 results in home migration and the migration notice is sent to P_2 and P_3 when P_0 releases the lock. The request of page x from P_2 results in home migration, but the migration notice did not send to P_1 and P_3 . When P_3 sends a request of page x to P_0 , since P_0 is the old home node of page x , it will forward the request to P_2 that results in a forwarded request.



The request forwarding problem occurs frequently for pages that have high degree of page sharing. To prevent the problem, for a page in the MIG state, if a new home node of a page receives a forwarding request from the old home node of

the page, it will change the state of the page to the I2 state. For a page in the I2 state, the home node performs the same method described in the INIT state to keep track of the copyset of the page. In an interval, a page in the I2 state will be changed to the FIX state if the home receives a forwarding request. At the second barrier of an interval, a page in the I2 state will be changed to the MIG state if the number of MIG copyset is larger than the number of FIX copyset. Otherwise, the state of a page will be changed from the I2 state to the FIX state.

III. PERFORMANCE EVALUATION

To evaluate the proposed protocol, we have implemented the proposed protocol along with the home-based protocol and the migratory home protocol on the CVM system and use MPI library for communication. We use four IBM eServers running Linux 2.4.20 to form an 8-processor cluster. Each IBM eServer has two 2.4 GHz processors, 1 GB memory and one Myrinet-2000 card. These machines are connected by a Myrinet-2000 switch. Eight application programs are used as test samples.

TABLE 1: APPLICATIONS, SYNCHRONIZATION USED IN APPLICATIONS, AND APPLICATION DESCRIPTIONS.

Program	Synchronization	Descriptions
Bucket Sort	Barriers, Locks	Bucket sorting on n integers
FFT	Barriers	Solving a partial differential equation
LU	Barriers	LU Factorization on $n \times n$ matrix
Merge Sort	Barriers, Locks	Merge sorting on n integers
Matrix Multiply	Barriers, Locks	Multiplying two $n \times n$ matrices
Radix Sort	Barriers	Radix sorting on n integers
Water-Nsquared	Barriers, Locks	simulating water molecules with a N-squared algorithm
Water Spatial	Barriers, Locks	simulating water molecules with a spatial algorithm

Table 1 shows the program names, synchronous methods, and the program descriptions of the eight application programs. The bucket sort program uses 256 buckets to sort n integers by p processors. It uses bubble sort to sort numbers in buckets. The FFT program solves a partial differential equation. The LU program is an LU factorization algorithm on $n \times n$ matrix. The merge sort program implements a 2-way

merge sort algorithm. The matrix multiply program multiplies two $n \times n$ matrix. The radix sort program implements a radix sort algorithm. The Water-Nsquared program simulates water molecules using an n-squared algorithm. The Water-Spatial program simulates water molecules using a spatial algorithm.

The experimental results of these eight application programs with various problem sizes and various numbers of nodes are shown in Figure 5. In Figure 5(a), the migratory home protocol and the adaptive migratory home protocol get little better performance than the fixed home protocol for the bucket sort program with 2-node. The reasons are two-fold. First, most of pages are in the MIG state. Second, a few number of request forwarding is detected after home migration, so almost no page is switch to the FIX state. Note that there is no request forwarding detected while running with two processors.

In Figure 5(b), the performance of the fixed home protocol is better than other two protocols for FFT program. There are two reasons. First, the numbers of diffs that can be saved are not many for home migration. Second, the number of page requests is increased a lot for the migratory home protocol and the adaptive migratory home protocol. In this test sample, most of pages are in the MIG state because a few number of request forwarding is detected.

In Figure 5(c), the migratory home protocol and the adaptive migratory home protocol outperform the fixed home protocol for the LU program. The reasons are two-fold. First, both migratory protocols reduce a lot of diff messages. Second, the number of page fault produced by both migratory protocols produce is a little higher than that of the fixed home protocol. In this case, most of pages are in the MIG state.

In Figure 5(d), the performance of the migratory home protocol and the adaptive migratory home protocol is better than the fixed home protocol for the merge sort program. The reasons are the same as those described for Figure 5(c). In this case, pages are in the MIG state. For this program, the amount of traffic is reduced about 1/6 by both migratory protocols.

In Figure 5(e), the performance of these three protocols is almost the same. There are two reasons. First, the synchronization between nodes is small. Second, the execution time is dominated by local execution. Therefore, the protocol effect does not affect this application.

In Figure 5(f), the performance of the fixed home protocol is a little better than other two protocols for the radix sort program. The reasons are three-fold. First, there is no request forwarding found for both migratory protocols and pages are in the MIG state. Second, the number of diff saved is not many. Third, page faults increase a lot for both migratory protocols.

In Figure 5(g), the performance of the fixed home protocol and the adaptive protocols protocol is better than the migratory home protocol for the water-Nsquared program. There are two reasons. First, although the migratory home protocol successfully eliminates ten thousands of diff messages, the number of request forwarding is very large. Second, the water-Nsquared program uses many locks, and most page faults occur within the critical sections of locks. The time of locking critical section and requesting a lock

increase a lot. In the adaptive migratory home protocol, states of most pages are changed from MIG to FIX because of detection of request forwarding, that is, the adaptive protocol can prevent the request forwarding.

In Figure 5(h), the performance of the fixed home protocol and the adaptive protocols protocol is better than the migratory home protocol for the water-spatial program. The reasons are the same as those described for Figure 5(g). Unlike the water-Nsquared program, the water-spatial program use less locks than the water-Nsquared program and the request-forwarding problem is more serious than that in the water-Nsquared program.

In general, the fixed home protocol deals with high degree of sharing pages well because of its low overheads on page fault and request forwarding. However, the performance of the fixed home protocol is not good enough for applications that have low degree of sharing and large number of diffs. On the other hand, the migratory home protocol has good performance for applications of low degree of sharing and of large number of diffs. But it is not suitable for applications with high degree of sharing pages. Since the adaptive migratory home protocol takes advantage of these two protocols, it can deal with most applications well.

IV. CONCLUSIONS

In this paper, we have proposed an adaptive migratory home protocol. From the experimental results, we have the following remarks.

Remark 1: The performance of the fixed home protocol is affected by the initial home assignment. It is very suitable for applications with high degree of page sharing, that is, most pages are shared by many nodes.

Remark 2: The migratory home protocol is benefit from the saving of diff creation and diff propagation of home migration. It is suitable for applications with low degree of page sharing, that is, most pages are used by only one node.

Remark 3: The adaptive migratory home protocol adapts between the fixed home protocol and the migratory home protocol. It has good performance for most of the tested applications. For the migratory access pattern, the behavior of most of pages is close to the migratory home protocol and benefits from the diff saving. For the shared access pattern, most of pages work like the fixed home protocol and can prevent request forwarding.

REFERENCES

- [1] Benny Wang-Leung Cheung, Cho-Li Wang and Kai Hwang, "A Migrating-Home Protocol for Implementing Scope Consistency Model on a Cluster of Workstations," *Journal of Information Science and Engineering*, 1999.
- [2] A. L. Cox, E. de Lara, Y. C. Hu, and W. Zwaenepoel, "A Performance Comparison of Homeless and Home-Based Lazy Release Consistency Protocols in Software Shared Memory," *Proceedings of the 5th IEEE Symposium on High-Performance Computer Architecture (HPCA-5)*, pages 279-283, January 1999.
- [3] Weiwu Hu, Weisong Shi and Zhimin Tang, "Home Migration in Home Based Software DSMs," *Proceedings of ACM 1st Workshop on Software DSM System*, (in Conjunction with ICS'99), Greece, June, 1999.
- [4] Peter J. Keleher, "Symmetry and Performance in Consistency Protocols," *International Conference on Supercomputing (ICS'99)*, June 1999.
- [5] Peter J. Keleher, "The Impact of Symmetry on Software Distributed Shared Memory," *Journal of Parallel and Distributed Computing (JPDC)*, 60(11): 1388-1419, 2000.
- [6] L. R. Monnerat and R. Bianchini, "Efficiently Adapting to Sharing Patterns in Software DSMs," *Proceedings of the 4th IEEE International Symposium on High-Performance Computer Architecture (HPCA98)*, Feb 1998.
- [7] M.C. Ng and W.F. Wong, "Adaptive Schemes for Home-based DSM Systems," *Proceedings of the 1999 Workshop on Software Distributed Shared Memory*, pages 13-20, June 1999.
- [8] Per Stenstrom, "A Survey of Cache Coherence Schemes for Multiprocessors," *IEEE Computer*, 23(6):12-24, June 1990.
- [9] Yuan Yuan Zhou, Liviu Iftode and Kai Li, "Performance Evaluation of Two Home-Based Lazy Release Consistency Protocols for Shared Virtual Memory Systems," *Proceedings of the 2nd Symposium on Operating Systems Design and Implementation*, October 1996.

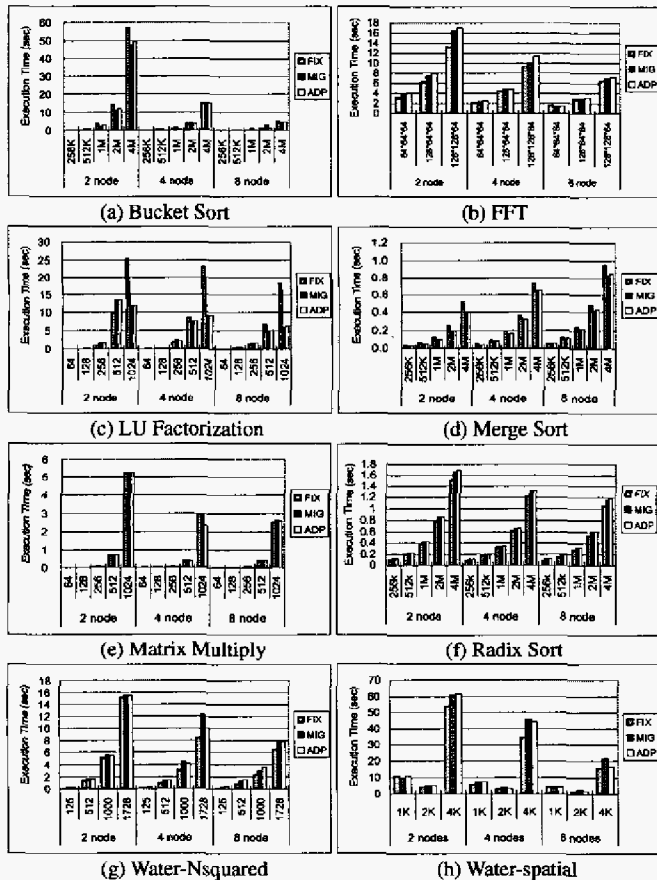


Figure 5: The execution time of application programs.